

Unit-2

Relational Model in DBMS

Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

Domain: It contains a set of atomic values that an attribute can take.

Attribute: It contains the name of a column in a particular table. Each attribute A_i must have a domain, $dom(A_i)$

Relational instance: In the relational database system, the relational instance is represented by a finite set of tuples. Relation instances do not have duplicate tuples.

Relational schema: A relational schema contains the name of the relation and name of all columns or attributes.

Relational key: In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

Example: STUDENT Relation

NAME	ROLL_NO	PHONE_NO	ADDRESS	AGE
Ram	14795	7305758992	Noida	24
Shyam	12839	9026288936	Delhi	35
Laxman	33289	8583287182	Gurugram	20
Mahesh	27857	7086819134	Ghaziabad	27
Ganesh	17282	9028 9i3988	Delhi	40

- In the given table, NAME, ROLL_NO, PHONE_NO, ADDRESS, and AGE are the attributes.
- The instance of schema STUDENT has 5 tuples.
- $t_3 = \langle \text{Laxman}, 33289, 8583287182, \text{Gurugram}, 20 \rangle$

Properties of Relations

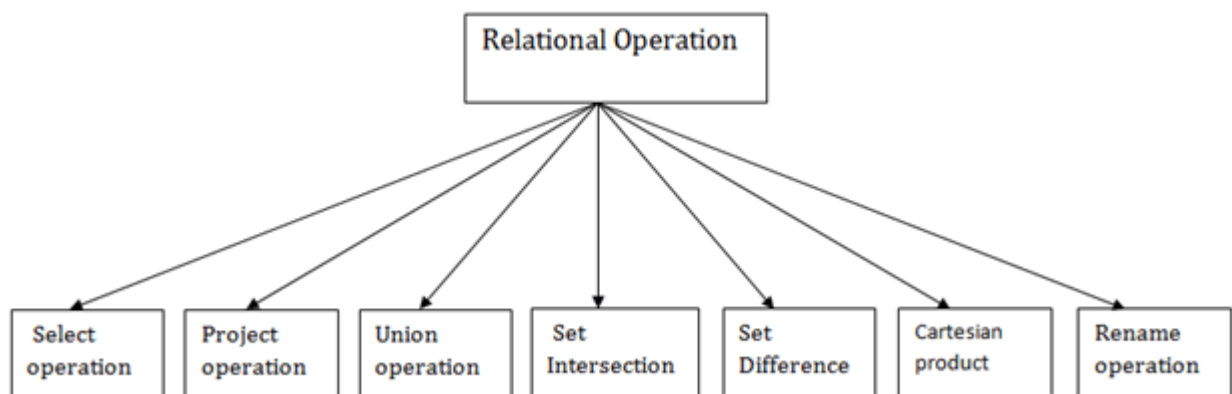
- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value
- Each attribute contains a distinct name
- Attribute domain has no significance

- tuple has no duplicate value
- Order of tuple can have a different sequence

Relational Algebra

Relational algebra is a procedural query language. It gives a step by step process to obtain the result of the query. It uses operators to perform queries.

Types of Relational operation



1. Select Operation:

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).

1. Notation: $\sigma_p(r)$

Where:

σ is used for selection
 r is used for relation
 p is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like $=, \neq, \geq, <, >, \leq$.

For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:

1. σ BRANCH_NAME="perryride" (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
- It is denoted by π .

1. Notation: π A1, A2, An (r)

Where

A1, A2, A3 is used as an attribute name of relation **r**.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison

Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Johnson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

1. Π NAME, CITY (CUSTOMER)

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn
Brooks	Brooklyn

3. Union Operation:

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by $\cup$.

1. Notation: $R \cup S$

A union operation must hold the following condition:

- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
Johnson	A-101
Smith	A-121
Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17
Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11
Williams	L-17

Input:

- Π CUSTOMER_NAME (BORROW) $\cup$ Π CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner

Jones
Lindsay
Jackson
Curry
Williams
Mayes

4. Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection $\cap$.

1. Notation: $R \cap S$

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. $\Pi \text{ CUSTOMER_NAME (BORROW)} \cap \Pi \text{ CUSTOMER_NAME (DEPOSITOR)}$

Output:

CUSTOMER_NAME
Smith
Jones

5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).

1. Notation: $R - S$

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. $\prod$ CUSTOMER_NAME (BORROW) - $\prod$ CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Jackson
Hayes
Willians
Curry

6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.

1. Notation: E X D

Example:

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Input:

1. EMPLOYEE X DEPARTMENT

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

7. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **ρ** (ρ).

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

1. ρ (STUDENT1, STUDENT)

Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by $\bowtie$.

Example:

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

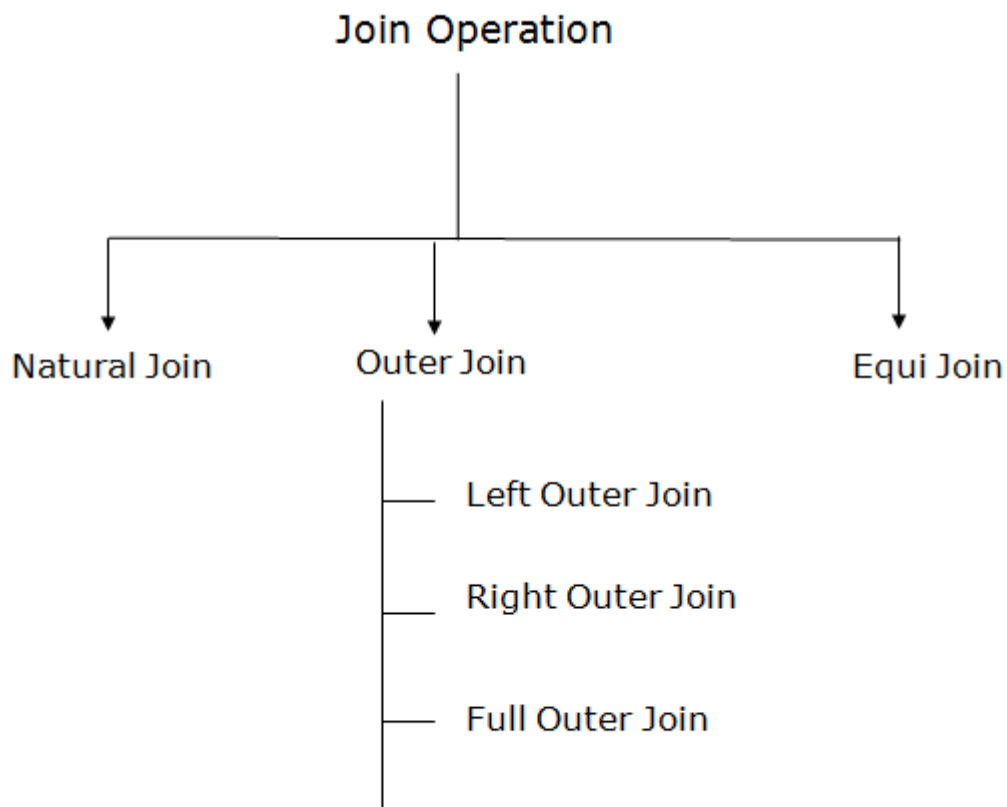
EMP_CODE	SALARY
101	50000
102	30000
103	25000

1. Operation: (EMPLOYEE ⋈ SALARY)

Result

EMP_CODE	EMP_NAME	SALARY
101	Stephan	50000
102	Jack	30000
103	Harry	25000

Types of Join operations:



1. Natural Join:

- A natural join is the set of tuples of all combinations in R and S that are equal on their common attribute names.
- It is denoted by $\bowtie$.

Example: Let's use the above EMPLOYEE table and SALARY table:

Input:

1. Π EMP_NAME, SALARY (EMPLOYEE $\bowtie$ SALARY)

Output:

EMP_NAME	SALARY
Stephan	50000
Jack	30000

Harry	25000
-------	-------

2. Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missing information.

Example:

EMPLOYEE

EMP_NAME	STREET	CITY
Ram	Civil line	Mumbai
Shyam	Park street	Kolkata
Ravi	M.G. Street	Delhi
Hari	Nehru nagar	Hyderabad

FACT_WORKERS

EMP_NAME	BRANCH	SALARY
Ram	Infosys	10000
Shyam	Wipro	20000
Kuber	HCL	30000
Hari	TCS	50000

Input:

1. (EMPLOYEE ⋈ FACT_WORKERS)

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru nagar	Hyderabad	TCS	50000

An outer join is basically of three types:

- a. Left outer join
- b. Right outer join
- c. Full outer join

a. Left outer join:

- Left outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE $\bowtie$ FACT_WORKERS

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL

b. Right outer join:

- Right outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS Relation

Input:

1. EMPLOYEE $\bowtie$ FACT_WORKERS

Output:

EMP_NAME	BRANCH	SALARY	STREET	CITY
Ram	Infosys	10000	Civil line	Mumbai
Shyam	Wipro	20000	Park street	Kolkata
Hari	TCS	50000	Nehru street	Hyderabad
Kuber	HCL	30000	NULL	NULL

c. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from both tables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S that have no matching tuples in R in their common attribute name.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

1. EMPLOYEE $\bowtie$ FACT_WORKERS

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL
Kuber	NULL	NULL	HCL	30000

3. Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data as per the equality condition. The equi join uses the comparison operator(=).

Example:

CUSTOMER RELATION

CLASS_ID	NAME
1	John
2	Harry
3	Jackson

PRODUCT

PRODUCT_ID	CITY
1	Delhi
2	Mumbai
3	Noida

Input:

1. CUSTOMER $\bowtie$ PRODUCT

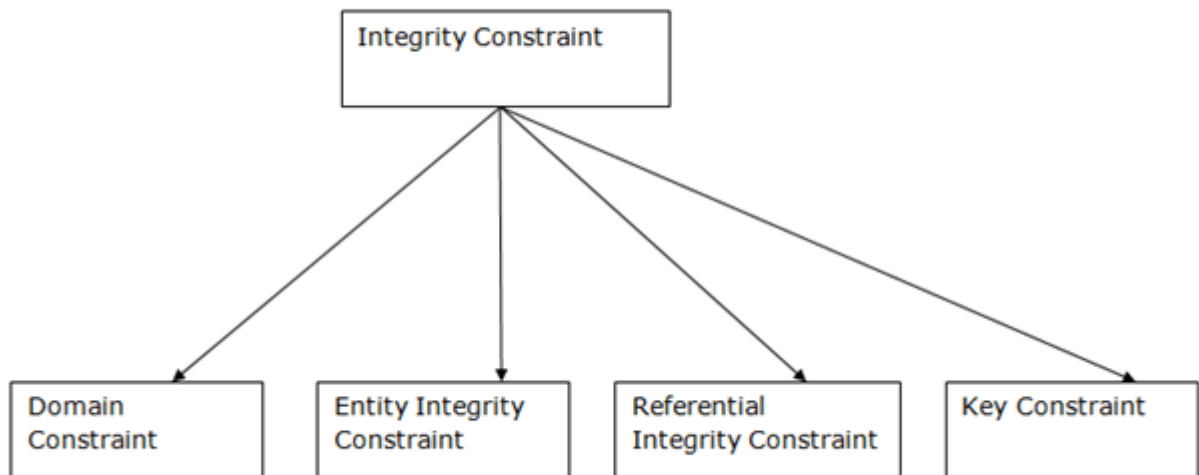
Output:

CLASS_ID	NAME	PRODUCT_ID	CITY
1	John	1	Delhi
2	Harry	2	Mumbai
3	Harry	3	Noida

Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.

- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

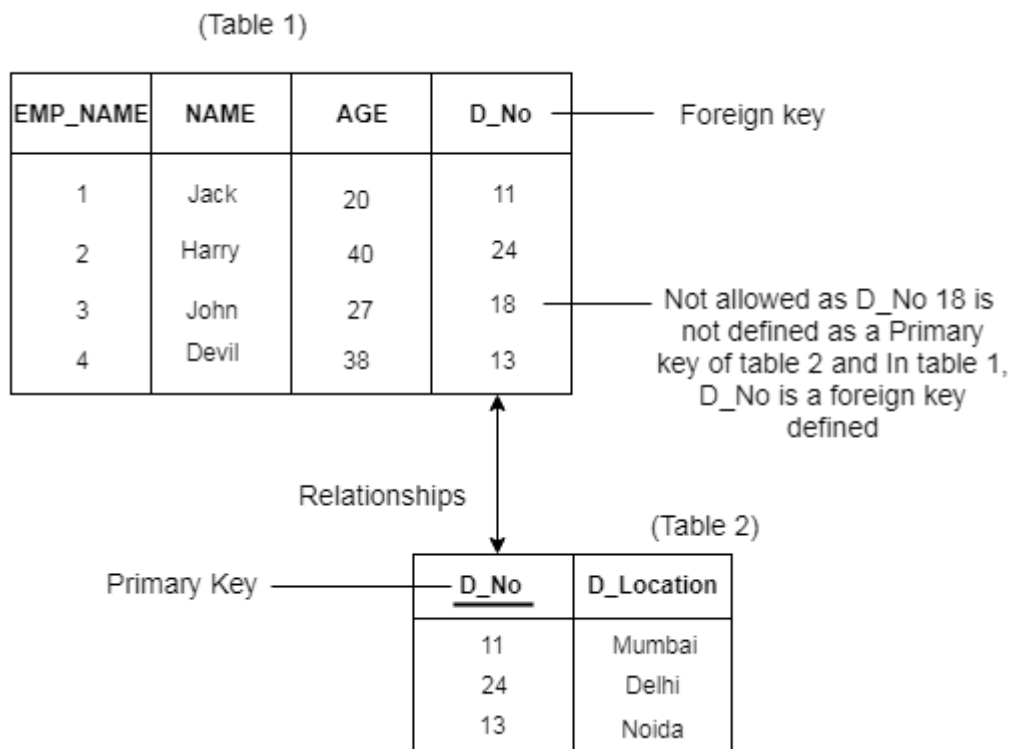
EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:



4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

Relational Calculus

There is an alternate way of formulating queries known as Relational Calculus. Relational calculus is a non-procedural query language. In the non-procedural query language, the user is concerned with the details of how to obtain the end results. The relational calculus tells what to do but never explains how to do. Most commercial relational languages are based on aspects of relational calculus including SQL-QBE and QUEL.

Why it is called Relational Calculus?

It is based on Predicate calculus, a name derived from branch of symbolic language. A predicate is a truth-valued function with arguments. On substituting values for the arguments, the function result in an expression called a proposition. It can be either true or false. It is a tailored version of a subset of the Predicate Calculus to communicate with the relational database.

Many of the calculus expressions involves the use of Quantifiers. There are two types of quantifiers:

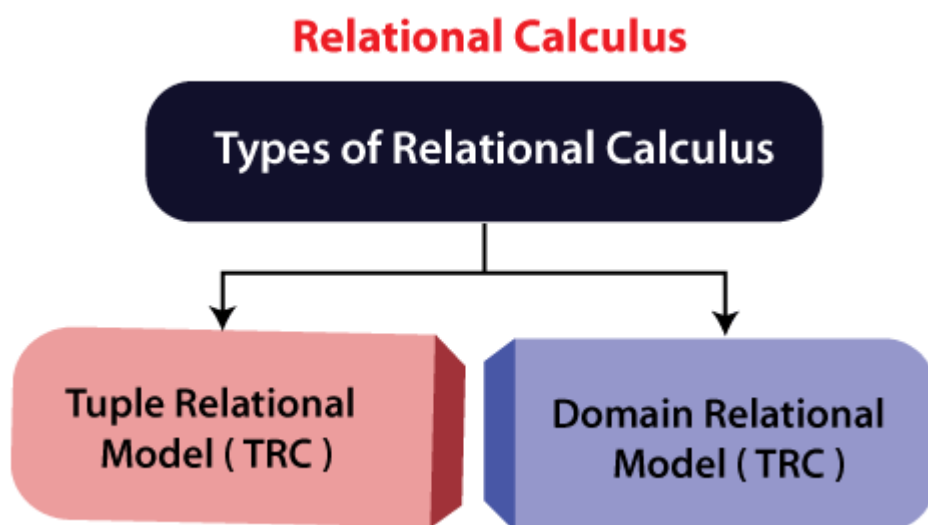
- **Universal Quantifiers:** The universal quantifier denoted by $\forall$ is read as for all which means that in a given set of tuples exactly all tuples satisfy a given condition.
- **Existential Quantifiers:** The existential quantifier denoted by $\exists$ is read as for all which means that in a given set of tuples there is at least one occurrences whose value satisfy a given condition.

Before using the concept of quantifiers in formulas, we need to know the concept of Free and Bound Variables.

A tuple variable t is bound if it is quantified which means that if it appears in any occurrences a variable that is not bound is said to be free.

Free and bound variables may be compared with global and local variable of programming languages.

Types of Relational calculus:



1. Tuple Relational Calculus (TRC)

It is a non-procedural query language which is based on finding a number of tuple variables also known as range variable for which predicate holds true. It describes the desired information without giving a specific procedure for obtaining that information. The tuple relational calculus is specified to select the tuples in a relation. In TRC, filtering variable uses the tuples of a relation. The result of the relation can have one or more tuples.

Notation:

A Query in the tuple relational calculus is expressed as following notation

1. $\{T \mid P(T)\}$ or $\{T \mid \text{Condition}(T)\}$

Where

T is the resulting tuples

P(T) is the condition used to fetch T.

For example:

1. $\{ T.\text{name} \mid \text{Author}(T) \text{ AND } T.\text{article} = \text{'database'} \}$

Output: This query selects the tuples from the AUTHOR relation. It returns a tuple with 'name' from Author who has written an article on 'database'.

TRC (tuple relation calculus) can be quantified. In TRC, we can use Existential ($\exists$) and Universal Quantifiers ($\forall$).

For example:

1. $\{ R \mid \exists T \in \text{Authors}(T.\text{article} = \text{'database'} \text{ AND } R.\text{name} = T.\text{name}) \}$

Output: This query will yield the same result as the previous one.

2. Domain Relational Calculus (DRC)

The second form of relation is known as Domain relational calculus. In domain relational calculus, filtering variable uses the domain of attributes. Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives $\wedge$ (and), $\vee$ (or) and $\neg$ (not). It uses Existential ($\exists$) and Universal Quantifiers ($\forall$) to bind the variable. The QBE or Query by example is a query language related to domain relational calculus.

Notation:

1. $\{ a_1, a_2, a_3, \dots, a_n \mid P(a_1, a_2, a_3, \dots, a_n) \}$

Where

a1, **a2** are attributes
P stands for formula built by inner attributes

For example:

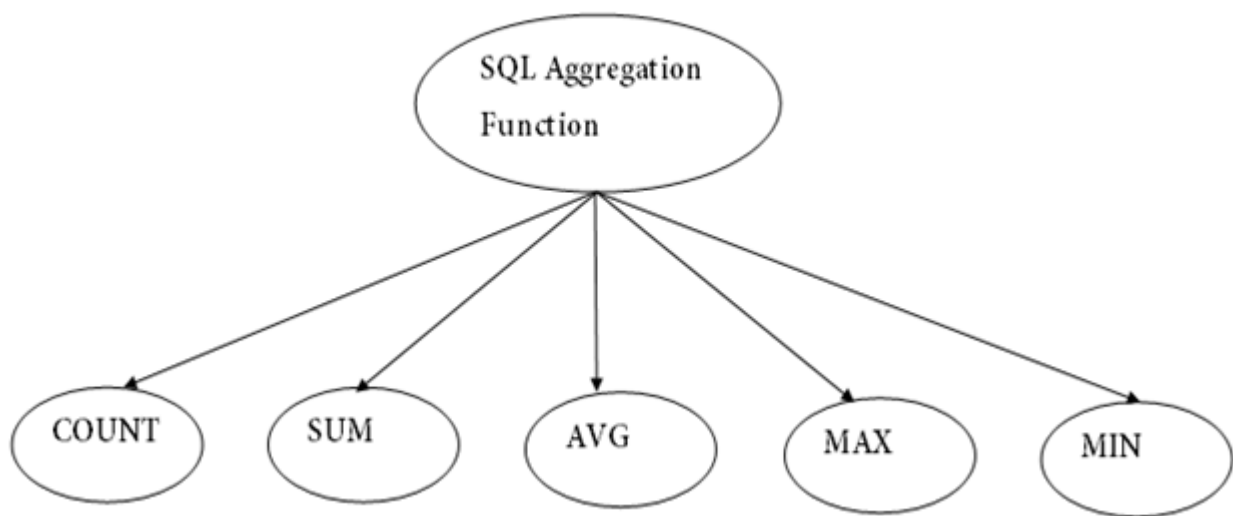
1. {< article, page, subject > | $\in \text{javatpoint} \wedge \text{subject} = \text{'database'}$ }

Output: This query will yield the article, page, and subject from the relational javatpoint, where the subject is a database.

SQL Aggregate Functions

- SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.
- It is also used to summarize the data.

Types of SQL Aggregation Function



1. COUNT FUNCTION

- COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.
- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax

1. COUNT(*)
2. or
3. COUNT([ALL|DISTINCT] expression)

Sample table:

PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50
Item10	Com3	4	30	120

Example: COUNT(SELECT COUNT(*))

1. FROM PRODUCT_MAST;

Output:

10

Example: COUNT with WHERE

1. SELECT COUNT(*)
2. FROM PRODUCT_MAST;
3. WHERE RATE>=20;

Output:

7

Example: COUNT() with DISTINCT

1. SELECT COUNT(DISTINCT COMPANY)
2. FROM PRODUCT_MAST;

Output:

```
3
```

Example: COUNT() with GROUP BY

1. SELECT COMPANY, COUNT(*)
2. FROM PRODUCT_MAST
3. GROUP BY COMPANY;

Output:

```
Com1  5
Com2  3
Com3  2
```

Example: COUNT() with HAVING

1. SELECT COMPANY, COUNT(*)
2. FROM PRODUCT_MAST
3. GROUP BY COMPANY
4. HAVING COUNT(*)>2;

Output:

```
Com1  5
Com2  3
```

2. SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

1. SUM()
2. or
3. SUM([ALL|DISTINCT] expression)

Example: SUM()

1. SELECT SUM(COST)
2. FROM PRODUCT_MAST;

Output:

Example: SUM() with WHERE

1. SELECT SUM(COST)
2. FROM PRODUCT_MAST
3. WHERE QTY>3;

Output:

320

Example: SUM() with GROUP BY

1. SELECT SUM(COST)
2. FROM PRODUCT_MAST
3. WHERE QTY>3
4. GROUP BY COMPANY;

Output:

Com1	150
Com2	170

Example: SUM() with HAVING

1. SELECT COMPANY, SUM(COST)
2. FROM PRODUCT_MAST
3. GROUP BY COMPANY
4. HAVING SUM(COST)>=170;

Output:

Com1	335
Com3	170

3. AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

1. AVG()
2. or
3. AVG([ALL|DISTINCT] expression)

Example:

1. SELECT AVG(COST)
2. FROM PRODUCT_MAST;

Output:

4. MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

1. MAX()
2. or
3. MAX([ALL|DISTINCT] expression)

Example:

1. SELECT MAX(RATE)
2. FROM PRODUCT_MAST;

5. MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

1. MIN()
2. or
3. MIN([ALL|DISTINCT] expression)

Example:

1. SELECT MIN(RATE)
2. FROM PRODUCT_MAST;

Output:

Teradata Built-In Functions

Teradata provides built-in functions, which are extensions to [SQL](#). The Built-in functions return information about the system.

Built-in functions are sometimes referred to as individual registers. It can be used anywhere that a literal can appear.

If a SELECT statement that contains a built-in function references a table name, then the result of the query contains one row for every row of the table that satisfies the search condition.

Some common built-in functions are listed below with examples, such as

S. No.	Function	Example
1	SELECT DATE;	Date- 2018/02/10
2	SELECT CURRENT_DATE;	Date- 2020/05/23
3	SELECT TIME;	Time- 09:02:00
4	SELECT CURRENT_TIME;	Time- 10:01:13
5	SELECT CURRENT_TIMESTAMP;	Current TimeStamp(6)- 2020-05-23 10:01:13.990000+00.00
6	SELECT DATABASE;	Database- TDUSER

SQL Sub Query

A Subquery is a query within another SQL query and embedded within the WHERE clause.

Important Rule:

- A subquery can be placed in a number of SQL clauses like WHERE clause, FROM clause, HAVING clause.
- You can use Subquery with SELECT, UPDATE, INSERT, DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.
- A subquery is a query within another query. The outer query is known as the main query, and the inner query is known as a subquery.
- Subqueries are on the right side of the comparison operator.
- A subquery is enclosed in parentheses.
- In the Subquery, ORDER BY command cannot be used. But GROUP BY command can be used to perform the same function as ORDER BY command.

1. Subqueries with the Select Statement

SQL subqueries are most frequently used with the Select statement.

Syntax

1. SELECT column_name
2. FROM table_name
3. WHERE column_name expression operator
4. (SELECT column_name from table_name WHERE ...);

Example

Consider the EMPLOYEE table have the following records:

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
4	Alina	29	UK	6500.00
5	Kathrin	34	Bangalore	8500.00
6	Harry	42	China	4500.00
7	Jackson	25	Mizoram	10000.00

The subquery with a SELECT statement will be:

1. SELECT *
2. FROM EMPLOYEE
3. WHERE ID IN (SELECT ID
4. FROM EMPLOYEE
5. WHERE SALARY > 4500);

This would produce the following result:

ID	NAME	AGE	ADDRESS	SALARY
4	Alina	29	UK	6500.00

5	Kathrin	34	Bangalore	8500.00
7	Jackson	25	Mizoram	10000.00

2. Subqueries with the INSERT Statement

- SQL subquery can also be used with the Insert statement. In the insert statement, data returned from the subquery is used to insert into another table.
- In the subquery, the selected data can be modified with any of the character, date functions.

Syntax:

1. INSERT INTO table_name (column1, column2, column3....)
2. SELECT *
3. FROM table_name
4. WHERE VALUE OPERATOR

Example

Consider a table EMPLOYEE_BKP with similar as EMPLOYEE.

Now use the following syntax to copy the complete EMPLOYEE table into the EMPLOYEE_BKP table.

1. INSERT INTO EMPLOYEE_BKP
2. SELECT * FROM EMPLOYEE
3. WHERE ID IN (SELECT ID
4. FROM EMPLOYEE);

3. Subqueries with the UPDATE Statement

The subquery of SQL can be used in conjunction with the Update statement. When a subquery is used with the Update statement, then either single or multiple columns in a table can be updated.

Syntax

1. UPDATE table
2. SET column_name = new_value
3. WHERE VALUE OPERATOR
4. (SELECT COLUMN_NAME
5. FROM TABLE_NAME
6. WHERE condition);

Example

Let's assume we have an EMPLOYEE_BKP table available which is backup of EMPLOYEE table. The given example updates the SALARY by .25 times in the EMPLOYEE table for all employee whose AGE is greater than or equal to 29.

1. UPDATE EMPLOYEE
2. SET SALARY = SALARY * 0.25
3. WHERE AGE IN (SELECT AGE FROM CUSTOMERS_BKP
4. WHERE AGE >= 29);

This would impact three rows, and finally, the EMPLOYEE table would have the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
4	Alina	29	UK	1625.00
5	Kathrin	34	Bangalore	2125.00
6	Harry	42	China	1125.00
7	Jackson	25	Mizoram	10000.00

4. Subqueries with the DELETE Statement

The subquery of SQL can be used in conjunction with the Delete statement just like any other statements mentioned above.

Syntax

1. DELETE FROM TABLE_NAME
2. WHERE VALUE OPERATOR
3. (SELECT COLUMN_NAME
4. FROM TABLE_NAME
5. WHERE condition);

Example

Let's assume we have an EMPLOYEE_BKP table available which is backup of EMPLOYEE table. The given example deletes the records from the EMPLOYEE table for all EMPLOYEE whose AGE is greater than or equal to 29.

1. DELETE FROM EMPLOYEE
2. WHERE AGE IN (SELECT AGE FROM EMPLOYEE_BKP
3. WHERE AGE >= 29);

This would impact three rows, and finally, the EMPLOYEE table would have the following records.

ID	NAME	AGE	ADDRESS	SALARY
1	John	20	US	2000.00
2	Stephan	26	Dubai	1500.00
3	David	27	Bangkok	2000.00
7	Jackson	25	Mizoram	10000.00

Introduction to SQL correlated subquery

Let's start with an example.

See the following employees table in the [sample database](#):

employees
* employee_id
first_name
last_name
email
phone_number
hire_date
job_id
salary
manager_id
department_id

The following query finds employees whose salary is greater than the average salary of all employees:

SELECT

FROM

WHERE

SELECT
AVG
FROM

Code language: SQL (Structured Query Language) (sql)

	employee_id	first_name	last_name	salary
▶	121	Adam	Fripp	8200.00
	103	Alexander	Hunold	9000.00
	109	Daniel	Faviet	9000.00
	114	Den	Raphaely	11000.00
	204	Hermann	Baer	10000.00
	177	Jack	Livingston	8400.00
	110	John	Chen	8200.00
	145	John	Russell	14000.00
	176	Jonathon	Taylor	8600.00
	146	Karen	Partners	13500.00
	102	Lex	De Haan	17000.00

In this example, the subquery is used in the [WHERE](#) clause. There are some points that you can see from this query:

First, you can execute the subquery that returns the [average](#) salary of all employees independently.

SELECT
AVG
FROM

Code language: SQL (Structured Query Language) (sql)

Second, the database system needs to evaluate the subquery only once.

Third, the outer query makes use of the result returned from the subquery. The outer query depends on the subquery for its value. However, the subquery does not depend on the outer query. Sometimes, we call this subquery a plain subquery.

Unlike a plain subquery, a correlated subquery is a subquery that uses the values from the outer query. Also, a correlated subquery may be evaluated once for each row selected by the outer query. Because of this, a query that uses a correlated subquery may be slow.

A correlated subquery is also known as a repeating subquery or a synchronized subquery.

SQL correlated subquery examples

Let's see few more examples of the correlated subqueries to understand them better.

SQL correlated subquery in the WHERE clause example

The following query finds all employees whose salary is higher than the average salary of the employees in their departments:

SELECT

```
FROM  
WHERE  
SELECT  
AVG  
FROM  
WHERE  
ORDER BY
```

Code language: SQL (Structured Query Language) (sql)

Here is the output:

	employee_id	first_name	last_name	salary	department_id
►	201	Michael	Hartstein	13000.00	2
	114	Den	Raphaely	11000.00	3
	121	Adam	Fripp	8200.00	5
	120	Matthew	Weiss	8000.00	5
	122	Payam	Kaufing	7900.00	5
	123	Shanta	Vollman	6500.00	5
	103	Alexander	Hunold	9000.00	6
	104	Bruce	Ernst	6000.00	6
	145	John	Russell	14000.00	8
	146	Karen	Partners	13500.00	8
	100	Steven	King	24000.00	9
	109	Daniel	Faviet	9000.00	10
	108	Nancy	Greenberg	12000.00	10

In this example, the outer query is:

```
SELECT
```

```
FROM
```

```
WHERE
```

Code language: SQL (Structured Query Language) (sql)

and the correlated subquery is:

```
SELECT  
  AVG  
FROM
```

```
WHERE
```

Code language: SQL (Structured Query Language) (sql)

For each employee, the database system has to execute the correlated subquery once to calculate the average salary of the employees in the department of the current employee.

SQL correlated subquery in the `SELECT` clause example

The following query returns the employees and the average salary of all employees in their departments:

```
SELECT
```

```
  SELECT  
    ROUND AVG      0  
  FROM  
  
  WHERE  
  
FROM  
  
  INNER JOIN  
    ON  
ORDER BY
```

Code language: SQL (Structured Query Language) (sql)

The output is:

	employee_id	first_name	last_name	department_name	salary	avg_salary_in_department
▶	205	Shelley	Higgins	Accounting	12000.00	10150
	206	William	Gietz	Accounting	8300.00	10150
	200	Jennifer	Whalen	Administration	4400.00	4400
	102	Lex	De Haan	Executive	17000.00	19333
	101	Neena	Kochhar	Executive	17000.00	19333
	100	Steven	King	Executive	24000.00	19333
	109	Daniel	Faviet	Finance	9000.00	8600
	111	Ismael	Sciarra	Finance	7700.00	8600
	110	John	Chen	Finance	8200.00	8600
	112	Jose Manuel	Urman	Finance	7800.00	8600
	113	Luis	Popp	Finance	6900.00	8600
	108	Nancy	Greenberg	Finance	12000.00	8600
	203	Susan	Mavris	Human Resources	6500.00	6500
	103	Alexander	Hunold	IT	9000.00	5760

For each employee, the database system has to execute the correlated subquery once to calculate the average salary by the employee's department.

SQL correlated subquery with EXISTS operator example

We often use a correlated subquery with the [EXISTS](#) operator. For example, the following query returns all employees who have no dependents:

SELECT

FROM

WHERE
NOT EXISTS SELECT

FROM

WHERE

ORDER BY

Code language: SQL (Structured Query Language) (sql)

The following picture shows the output:

	employee_id	first_name	last_name
▶	121	Adam	Fripp
	193	Britney	Everett
	179	Charles	Johnson
	126	Irene	Mikkilineni
	177	Jack	Livingston
	178	Kimberely	Grant
	120	Matthew	Weiss
	122	Payam	Kaufling
	192	Sarah	Bell
	123	Shanta	Vollman

In this tutorial, you have learned about the SQL correlated subquery and how to apply it to form a complex query.

The SQL GROUP BY Statement

The **GROUP BY** statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The **GROUP BY** statement is often used with aggregate functions (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**) to group the result-set by one or more columns.

GROUP BY Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin

2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå

ADVERTISEMENT

SQL GROUP BY Examples

The following SQL statement lists the number of customers in each country:

Example [Get your own SQL Server](#)

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

[Try it Yourself »](#)

The following SQL statement lists the number of customers in each country, sorted high to low:

Example

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

[Try it Yourself »](#)

Demo Database

Below is a selection from the "Orders" table in the Northwind sample database:

OrderID	CustomerID	EmployeeID	OrderDate
10248	90	5	1996-07-04
10249	81	6	1996-07-05
10250	34	4	1996-07-08

And a selection from the "Shippers" table:

ShipperID	ShipperName
1	Speedy Express
2	United Package
3	Federal Shipping

GROUP BY With JOIN Example

The following SQL statement lists the number of orders sent by each shipper:

Example

```
SELECT Shippers.ShipperName, COUNT(Orders.OrderID) AS NumberOfOrders FROM Orders
LEFT JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
GROUP BY ShipperName;
```

The SQL HAVING Clause

The **HAVING** clause was added to SQL because the **WHERE** keyword cannot be used with aggregate functions.

HAVING Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå

SQL HAVING Examples

The following SQL statement lists the number of customers in each country. Only include countries with more than 5 customers:

Example [Get your own SQL Server](#)

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;
```

[Try it Yourself »](#)

The following SQL statement lists the number of customers in each country, sorted high to low (Only include countries with more than 5 customers):

Example

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;
```

The SQL HAVING Clause

The **HAVING** clause was added to SQL because the **WHERE** keyword cannot be used with aggregate functions.

HAVING Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Demo Database

Below is a selection from the "Customers" table in the Northwind sample database:

CustomerID	CustomerName	ContactName	Address	City
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin

2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	México D.F.
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå

SQL HAVING Examples

The following SQL statement lists the number of customers in each country. Only include countries with more than 5 customers:

Example [Get your own SQL Server](#)
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;

[Try it Yourself »](#)

The following SQL statement lists the number of customers in each country, sorted high to low (Only include countries with more than 5 customers):

Example
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) **DESC**;

The SQL ANY and ALL Operators

The **ANY** and **ALL** operators allow you to perform a comparison between a single column value and a range of other values.

The SQL ANY Operator

The **ANY** operator:

- returns a boolean value as a result
- returns TRUE if ANY of the subquery values meet the condition

ANY means that the condition will be true if the operation is true for any of the values in the range.

ANY Syntax

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
(SELECT column_name
FROM table_name
WHERE condition);
```

Note: The *operator* must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

The SQL ALL Operator

The **ALL** operator:

- returns a boolean value as a result
- returns TRUE if ALL of the subquery values meet the condition
- is used with **SELECT**, **WHERE** and **HAVING** statements

ALL means that the condition will be true only if the operation is true for all values in the range.

ALL Syntax With SELECT

```
SELECT ALL column_name(s)
FROM table_name
WHERE condition;
```

ALL Syntax With WHERE or HAVING

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL
(SELECT column_name
FROM table_name
WHERE condition);
```

Note: The *operator* must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

Demo Database

Below is a selection from the "**Products**" table in the Northwind sample database:

ProductID	ProductName	SupplierID	CategoryID	Unit
1	Chais	1	1	10 boxes x
2	Chang	1	1	24 - 12 oz b
3	Aniseed Syrup	1	2	12 - 550 m
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz ja
5	Chef Anton's Gumbo Mix	2	2	36 boxes
6	Grandma's Boysenberry Spread	3	2	12 - 8 oz ja
7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pk
8	Northwoods Cranberry Sauce	3	2	12 - 12 oz j
9	Mishi Kobe Niku	4	6	18 - 500 g

And a selection from the "**OrderDetails**" table:

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
4	10249	14	9
5	10249	51	40
6	10250	41	10
7	10250	51	35
8	10250	65	15
9	10251	22	6
10	10251	57	15

ADVERTISEMENT

SQL ANY Examples

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity equal to 10 (this will return TRUE because the Quantity column has some values of 10):

Example [Get your own SQL Server](#)

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
  (SELECT ProductID
   FROM OrderDetails
   WHERE Quantity = 10);
```

[Try it Yourself »](#)

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity larger than 99 (this will return TRUE because the Quantity column has some values larger than 99):

Example

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
  (SELECT ProductID
   FROM OrderDetails
   WHERE Quantity > 99);
```

[Try it Yourself »](#)

The following SQL statement lists the ProductName if it finds ANY records in the OrderDetails table has Quantity larger than 1000 (this will return FALSE because the Quantity column has no values larger than 1000):

Example

```
SELECT ProductName
FROM Products
WHERE ProductID = ANY
  (SELECT ProductID
   FROM OrderDetails
   WHERE Quantity > 1000);
```

The SQL EXISTS Operator

The **EXISTS** operator is used to test for the existence of any record in a subquery.

The **EXISTS** operator returns TRUE if the subquery returns one or more records.

EXISTS Syntax

```
SELECT column_name(s)
FROM table_name
```

WHERE EXISTS

(**SELECT** *column_name* **FROM** *table_name* **WHERE** *condition*);

Demo Database

Below is a selection from the "Products" table in the Northwind sample database:

ProductID	ProductName	SupplierID	CategoryID	Unit
1	Chais	1	1	10 boxes x 2
2	Chang	1	1	24 - 12 oz b
3	Aniseed Syrup	1	2	12 - 550 ml
4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jar
5	Chef Anton's Gumbo Mix	2	2	36 boxes

And a selection from the "Suppliers" table:

SupplierID	SupplierName	ContactName	Address	City
1	Exotic Liquid	Charlotte Cooper	49 Gilbert St.	London
2	New Orleans Cajun Delights	Shelley Burke	P.O. Box 78934	New Orleans

3	Grandma Kelly's Homestead	Regina Murphy	707 Oxford Rd.	Ann Arbor
4	Tokyo Traders	Yoshi Nagase	9-8 Sekimai Musashino-shi	Tokyo

SQL EXISTS Examples

The following SQL statement returns TRUE and lists the suppliers with a product price less than 20:

Example [Get your own SQL Server](#)

```
SELECT SupplierName
FROM Suppliers
WHERE EXISTS (SELECT ProductName FROM Products WHERE Products.SupplierID =
Suppliers.supplierID AND Price < 20);
```

[Try it Yourself »](#)

The following SQL statement returns TRUE and lists the suppliers with a product price equal to 22:

Example

```
SELECT SupplierName
FROM Suppliers
WHERE EXISTS (SELECT ProductName FROM Products WHERE Products.SupplierID =
Suppliers.supplierID AND Price = 22);
```

What are the Different Types of SQL Views

What are the Different Types of SQL Views:

What exactly is SQL?

SQL stands for Structured Query Language, and it is a programming language used to store, manipulate, and retrieve data from a database system.

SQL is the database industry standard. [SQL](#) is the standard database dialect used by all Relational Database Management Systems (RDMS) such as MySQL, MS Access, Oracle, Sybase, Informix, Postgres, and SQL.

They also speak in distinct dialects, such as:

- MS SQL Server employs T-SQL.
- JET SQL (native format) is used by MS Access
- PL/SQL is used by Oracle

Table of Content:

[1\) What is View in SQL](#)

[2\) Types of View in SQL](#)

[3\) Syntax of View](#)

[4\) Difference between Complex and Simple View](#)

[5\) What is the purpose of SQL?](#)

What is View in SQL: A view is a little over than a SQL declaration with a name that is saved in the [database](#). A view is essentially a table inside the form of a preset SQL query.

A view can include all of the rows in a table or only a subset of the rows in a table. A view could be built from one or more tables, depending on the SQL query used to build the view.

Views, a form of the virtual table, enable users to achieve the following:

- Data from several tables may be summarized and utilized to make reports.
- Data should be structured in a way that consumers or groups of users find natural or simple.

Types of View in SQL:

In SQL Server, there really are 2 kinds of views: system-defined views and user-defined views.

A **Simple View** is a view that is constructed on a separate table. In simple views, we can only execute basic SQL actions. That is, we cannot execute analytical and aggregation operations in basic views via groups, sets, and so on. We can certainly conduct insert, modify, and delete straight from a basic view, but the main key columns must be present in the view.

Perspective on Dynamic Management: These were added to SQL Server in 2005. Through these views, the administrator may obtain information about the server state in order to diagnose issues, track the health of the test server, and modify performance. The Database-scoped Dynamic Management View is saved in each database, but the Server-scoped Dynamic Management View is only saved in the Master database.

The **Information Schema** database is an ANSI standard collection of views that may be found in both SQL Server and [MySQL](#). Other database management systems have either the same or comparable database implementation. It gives key sectors to database and object information (tables, restrictions, processes, views...) maintained on the client.

You might simply use this information to:

- Examine the contents of the server and/or the database.
- As a result, this database may be quite handy in some circumstances, particularly if you are in a DBA job.
- Automated processes and wrote some complicated code.
- Examine to ensure that everything is as it should be.

View Catalog: These are employed to return data from the SQL server. Catalog views are a quick and easy method to get, show, and alter custom types of information. Unfortunately, they do not provide

any detail on storage, duplicate, or maintenance plans, amongst many other things. Such views are used to retrieve database metadata, and the names and field names are descriptive, making it easier for a user to ask what is anticipated.

User-Defined Views: These are all the perspectives that users specify. User Defined Views are classified into two kinds: Simple Views and Complex Views.

Indexed Views: An indexed view is created when you construct a distinct principal theme on a consumer view. It increases query performance for queries that combine a large number of rows. They are ineffective in situations when the data is regularly changed.

Syntax of View:

To construct data views, just use the Construct VIEW method. A single table, numerous tables, or maybe another view can be used to construct a view. To build a view, a client must have the necessary system rights for the selected project.

The fundamental **syntax for CREATE VIEW** is as tries to follow:

```
CREATE VIEW view_name AS
SELECT column 1, column 2...
FROM table_name
WHERE [condition];
```

Difference between Complex and Simple View:

Complex View:

- Contains over one base record or is built from many tables.
- DML procedures can not be always done via a sophisticated view.
- Because there are numerous tables participating in a complicated view, generic connections including a join conditional, group by, or an order by clause must be used.
- It can have groups by, distinct, pseudo columns like row num, and expression-defined fields.

Simple View:

- Contains just one base table or is made up of only one table.
- DML operations might be carried out using a basic view.
- Simple views do not have grouped by, distinct, pseudo columns like row num, or expression-defined columns.

What is the purpose of SQL?

Following are a few strong arguments for why you should use SQL.

- SQL allows you to build and delete databases and tables.
- It helps with descriptive statistics.
- It enables you to specify and manipulate a database's data.
- It enables users to have access to information in the RDBMS system.
- Tables, methods, and views may all have permissions configured.

SQL (Structured Query Language) is a computer language that is used to store, alter, and retrieve information from a database.

The database accepted practice is SQL. SQL is the database dialect that is used by all Relational Database Management Systems (RDMS), including Oracle, MySQL, Sybase, MS Access, Informix, Postgres, and SQL.

TCL Commands in SQL

- In SQL, TCL stands for **Transaction control language**.
- A single unit of work in a database is formed after the consecutive execution of commands is known as a transaction.
- There are certain commands present in SQL known as TCL commands that help the user manage the transactions that take place in a database.
- **COMMIT**, **ROLLBACK** and **SAVEPOINT** are the most commonly used TCL commands in SQL.

Now let us take a deeper dive into the TCL commands of SQL with the help of examples. All the queries in the examples will be written using the MySQL database.

1. COMMIT

COMMIT command in SQL is used to save all the transaction-related changes permanently to the disk. Whenever DDL commands such as INSERT, UPDATE and DELETE are used, the changes made by these commands are permanent only after closing the current session. So before closing the session, one can easily roll back the changes made by the DDL commands. Hence, if we want the changes to be saved permanently to the disk without closing the session, we will use the commit command.

Syntax:

1. **COMMIT;**

Example:

We will select an existing database, i.e., school.

1. `mysql> USE school;`

```
MySQL 5.5 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.5.62 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement

mysql> USE school;
Database changed
```

To create a table named `t_school`, we will execute the following query:

1. `mysql> CREATE TABLE t_school(ID INT, School_Name VARCHAR(40), Number_Of_Students INT, Number_Of_Teachers INT, Number_Of_Classrooms INT, EmailID VARCHAR(40));`

```
mysql> CREATE TABLE t_school(ID INT, School_Name VARCHAR(40), Number_Of_Students
Classrooms INT, EmailID VARCHAR(40));
Query OK, 0 rows affected (0.24 sec)
```

BEGIN / START TRANSACTION command is used to start the transaction.

1. `mysql> START TRANSACTION;`

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

Now, we will execute the following query to insert multiple records at the same time in the `t_school` table.

1. mysql> **INSERT INTO** t_school(ID, School_Name, Number_Of_Students, Number_Of_Teachers, Number_Of_Classrooms, EmailID) **VALUES**(1, "Boys Town Public School", 1000, 80, 12, "btps15@gmail.com"), (2, "Guru Govind Singh Public School", 800, 35, 15, "ggps25@gmail.com"), (3, "Delhi Public School", 1200, 30, 10, "dps101@gmail.com"), (4, "Ashoka Universal School", 1110, 40, 40, "aus17@gmail.com"), (5, "Calibers English Medium School", 9000, 31, 50, "cems@gmail.com");

```
mysql> INSERT INTO t_school(ID, School_Name, Number_Of_Students, Number_Of_Teachers, Number_Of_Classrooms, EmailID) VALUES(1, "Boys Town Public School", 1000, 80, 12, "btps15@gmail.com"), (2, "Guru Govind Singh Public School", 800, 35, 15, "ggps25@gmail.com"), (3, "Delhi Public School", 1200, 30, 10, "dps101@gmail.com"), (4, "Ashoka Universal School", 1110, 40, 40, "aus17@gmail.com"), (5, "Calibers English Medium School", 9000, 31, 50, "cems@gmail.com");
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

We will now execute the SELECT query to verify the execution of the INSERT INTO query executed above.

1. mysql> **SELECT *FROM** t_school;

After executing the SELECT query on the t_school table, you will get the following output:

ID	School_Name	Number_Of_Students	Number_Of_Teachers	Number_Of_Classrooms
1	Boys Town Public School	1000	80	12
2	Guru Govind Singh Public School	800	35	15
3	Delhi Public School	1200	30	10
4	Ashoka Universal School	1110	40	40
5	Calibers English Medium School	9000	31	50

The output of the SELECT query shows that all the records are inserted successfully.

We will execute the COMMIT command to save the results of the operations carried on the t_school table.

1. mysql> **COMMIT**;

```
mysql> COMMIT;
Query OK, 0 rows affected (0.03 sec)
```

Autocommit is by default enabled in MySQL. To turn it off, we will set the value of autocommit as 0.

1. mysql> **SET** autocommit = 0;

```
mysql> SET autocommit = 0;
Query OK, 0 rows affected (0.08 sec)
```

MySQL, by default, commits every query the user executes. But if the user wishes to commit only the specific queries instead of committing every query, then turning off the autocommit is useful.

2. SAVEPOINT

We can divide the database operations into parts. For example, we can consider all the insert related queries that we will execute consecutively as one part of the transaction and the delete command as the other part of the transaction. Using the SAVEPOINT command in SQL, we can save these different parts of the same transaction using different names. ***For example***, we can save all the insert related queries with the savepoint named INS. To save all the insert related queries in one savepoint, we have to execute the SAVEPOINT query followed by the savepoint name after finishing the insert command execution.

Syntax:

1. SAVEPOINT savepoint_name;

3. ROLLBACK

While carrying a transaction, we must create savepoints to save different parts of the transaction. According to the user's changing requirements, he/she can roll back the transaction to different savepoints. *Consider a scenario:* We have initiated a transaction followed by the table creation and record insertion into the table. After inserting records, we have created a savepoint INS. Then we executed a delete query, but later we thought that mistakenly we had removed the useful record. Therefore in such situations, we have an option of rolling back our transaction. In this case, we have to roll back our transaction using the *ROLLBACK* command to the savepoint INS, which we have created before executing the DELETE query.

Syntax:

1. **ROLLBACK TO** savepoint_name;

Examples to understand the SAVEPOINT and ROLLBACK commands:

Example 1:

We will select an existing database, i.e., school.

1. mysql> USE school;

```
MySQL 5.5 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.5.62 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement

mysql> USE school;
Database changed
```

To create a table named t_school, we will execute the following query:

1. mysql> **CREATE TABLE** t_school(ID **INT**, School_Name **VARCHAR**(40), Number_Of_Students **INT**, Number_Of_Teachers **INT**, Number_Of_Classrooms **INT**, EmailID **VARCHAR**(40));

```
mysql> CREATE TABLE t_school(ID INT, School_Name VARCHAR(40), Number_Of_Students INT, Number_Of_Teachers INT, Number_Of_Classrooms INT, EmailID VARCHAR(40));
Query OK, 0 rows affected (0.24 sec)
```

Now, we will execute the following query to insert multiple records at the same time in the t_school table.

1. mysql> **INSERT INTO** t_school(ID, School_Name, Number_Of_Students, Number_Of_Teachers, Number_Of_Classrooms, EmailID) **VALUES**(1, "Boys Town Public School", 1000, 80, 12, "btps15@gmail.com"), (2, "Guru Govind Singh Public School", 800, 35, 15, "ggps25@gmail.com"), (3, "Delhi Public School", 1200, 30, 10, "dps101@gmail.com"), (4, "Ashoka Universal School", 1110, 40, 40, "aus17@gmail.com"), (5, "Calibers English Medium School", 9000, 31, 50, "cems@gmail.com");

```
mysql> INSERT INTO t_school(ID, School_Name, Number_Of_Students, Number_Of_Teachers, Number_Of_Classrooms, EmailID) VALUES(1, "Boys Town Public School", 1000, 80, 12, "btps15@gmail.com"), (2, "Guru Govind Singh Public School", 800, 35, 15, "ggps25@gmail.com"), (3, "Delhi Public School", 1200, 30, 10, "dps101@gmail.com"), (4, "Ashoka Universal School", 1110, 40, 40, "aus17@gmail.com"), (5, "Calibers English Medium School", 9000, 31, 50, "cems@gmail.com");
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

We will now execute the SELECT query to verify the execution of the INSERT INTO query executed above.

1. mysql> **SELECT *FROM** t_school;

After executing the SELECT query on the t_school table, you will get the following output:

ID	School_Name	Number_Of_Students	Number_Of_Teachers	Number_Of_Class
1	Boys Town Public School 1000	80	12	btps15@gmail.com
2	Guru Govind Singh Public School	800	35	15
3	Delhi Public School	1200	30	10
4	Ashoka Universal School	1110	40	40
5	Calibers English Medium School	9000	31	50

The output of the SELECT query shows that all the records are inserted successfully.

BEGIN / START TRANSACTION command is used to start the transaction.

1. mysql> **START TRANSACTION;**

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

As we know, the SAVEPOINT command in SQL is used to save the different parts of the same transaction using different names. Consider till this point as one part of our transaction. We will save this part using a savepoint named Insertion.

1. mysql> **SAVEPOINT** Insertion;

```
mysql> SAVEPOINT Insertion;
Query OK, 0 rows affected (0.00 sec)
```

Now, we will execute the update command on the t_school table to set the Number_Of_Students as 9050 for the record with ID 5.

1. mysql> **UPDATE** t_school **SET** Number_Of_Students = 9050 **WHERE** ID = 5;

```
mysql> UPDATE t_school SET Number_Of_Students = 9050 WHERE ID = 5;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

To verify that the record with ID 5 now has the Number_Of_Students as 9050, we will execute the SELECT query.

1. mysql> **SELECT *FROM** t_school;

After executing the SELECT query on the t_school table, you will get the following output:

ID	School_Name	Number_Of_Students	Number_Of_Teachers	Number_Of_Class
1	Boys Town Public School	1000	80	12
2	Guru Govind Singh Public School	800	35	15
3	Delhi Public School	1200	30	10
4	Ashoka Universal School	1110	40	40
5	Calibers English Medium School	9050	31	50

The output of the SELECT query shows that the record with ID 5 is updated successfully.

Consider the update operation as one part of our transaction. We will save this part using a savepoint named Updation.

1. mysql> SAVEPOINT Updation;

```
mysql> SAVEPOINT Updation;
Query OK, 0 rows affected (0.00 sec)
```

Suddenly, our requirement changed, and we realized that we had updated a record that was not supposed to be. In such a scenario, we need to roll back our transaction to the savepoint, which was created prior to the execution of the UPDATE command.

1. mysql> **ROLLBACK TO** Insertion;

```
mysql> ROLLBACK TO Insertion;
Query OK, 0 rows affected (0.09 sec)
```

We didn't need the updation carried on the record. Hence, we have rolled back to the savepoint named Insertion.

For confirming that we have got the same t_school table that we had before carrying out the updation operation, we will again execute the SELECT query.

1. mysql> **SELECT *FROM** t_school;

ID	School_Name	Number_Of_Students	Number_Of_Teachers	Number_Of_Class
1	Boys Town Public School	1000	80	12
2	Guru Govind Singh Public School	800	35	15
3	Delhi Public School	1200	30	10
4	Ashoka Universal School	1110	40	40
5	Calibers English Medium School	9000	31	50

The SELECT query output confirms that the transaction is now successfully rolled back to the savepoint 'Insertion'.

Example 2:

We will select an existing database, i.e., bank.

1. mysql> USE bank;

```
MySQL 5.5 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 5.5.62 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement

mysql> USE bank;
Database changed
```

To create a table named customer, we will execute the following query:

1. mysql> **CREATE TABLE** customer(Customer_ID **INT PRIMARY KEY**, Name **VARCHAR**(20), Age **INT**, Salary **INT**, Salary_BankAccount **VARCHAR**(20));

```
mysql> CREATE TABLE customer(Customer_ID INT PRIMARY KEY, Name VARCHAR(20), A
HAR(20));
Query OK, 0 rows affected (0.17 sec)
```

Now, we will execute the following query to insert multiple records at the same time in the customer table.

1. mysql> **INSERT INTO** customer(Customer_ID, Name, Age, Salary, Salary_BankAccount) **VALUE S**(1, "Aryan Jain", 51, 56000, "SBI"), (2, "Arohi Dixit", 21, 25000, "Axis"), (3, "Vineet Garg", 24, 31000, "ICICI"), (4, "Anuja Sharma", 26, 49000, "HDFC"), (5, "Deepak Kohli", 28, 65000, "SBI");

```
mysql> INSERT INTO customer(Customer_ID, Name, Age, Salary, Salary_BankAccount
), (2, "Arohi Dixit", 21, 25000, "Axis"), (3, "Vineet Garg", 24, 31000, "ICICI
, (5, "Deepak Kohli", 28, 65000, "SBI");
Query OK, 5 rows affected (0.07 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

We will now execute the SELECT query to verify the execution of the INSERT INTO query executed above.

1. mysql> **SELECT *FROM** customer;

After executing the SELECT query on the t_school table, you will get the following output:

Customer_ID	Name	Age	Salary	Salary_BankAccount
1	Aryan Jain	51	56000	SBI
2	Arohi Dixit	21	25000	Axis
3	Vineet Garg	24	31000	ICICI
4	Anuja Sharma	26	49000	HDFC
5	Deepak Kohli	28	65000	SBI

The output of the SELECT query shows that all the records are inserted successfully.

BEGIN / START TRANSACTION command is used to start the transaction.

1. mysql> START TRANSACTION;

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

As we know, the SAVEPOINT command in SQL is used to save the different parts of the same transaction using different names. Consider till this point as one part of our transaction. We will save this part using a savepoint named Insertion.

1. mysql> SAVEPOINT Insertion;

```
mysql> SAVEPOINT Insertion;
Query OK, 0 rows affected (0.00 sec)
```

We will execute the delete command on the customer table to remove the record with ID 5.

1. mysql> DELETE FROM customer WHERE Customer_ID = 5;

```
mysql> DELETE FROM customer WHERE Customer_ID = 5;
Query OK, 1 row affected (0.00 sec)
```

We will execute the SELECT query to verify that the record with ID 5 has been removed.

1. mysql> SELECT *FROM customer;

Customer_ID	Name	Age	Salary	Salary_BankAccount
1	Aryan Jain	51	56000	SBI
2	Arohi Dixit	21	25000	Axis
3	Vineet Garg	24	31000	ICICI
4	Anuja Sharma	26	49000	HDFC

The output of the SELECT query shows that the record with ID 5 is removed successfully.

Consider the delete operation as one part of our transaction. We will save this part using a savepoint named Deletion.

1. mysql> SAVEPOINT Deletion;

```
mysql> SAVEPOINT Deletion;
Query OK, 0 rows affected (0.00 sec)
```

Suddenly, our requirement changed, and we realized that we had deleted a record that was not supposed to be. In such a scenario, we need to roll back our transaction to the savepoint, which was created prior to the execution of the DELETE command.

1. mysql> **ROLLBACK TO** Insertion;

We didn't need the deletion carried on the record. Hence, we have rolled back to the savepoint named Insertion.

For confirming that we have got the same customer table that we had before carrying out the deletion operation, we will again execute the SELECT query.

1. mysql> **SELECT *FROM** customer;

Customer_ID	Name	Age	Salary	Salary_BankAccount
1	Aryan Jain	51	56000	SBI
2	Arohi Dixit	21	25000	Axis
3	Vineet Garg	24	31000	ICICI
4	Anuja Sharma	26	49000	HDFC
5	Deepak Kohli	28	65000	SBI

The SELECT query output confirms that the transaction is now successfully rolled back to the savepoint 'Insertion'.